FCRM Workbench — Build Runbook (10 Agents, from Zero)

2026-09-17 · @Nathan Myridius

Step-by-step guide for four developers new to AI agents to build the Risk Assessment Workbench's ten modules, starting from an empty repo and a Dockerized Ubuntu environment.

0. Prerequisites (every developer, before writing any agent code)

Docker Desktop is already installed. Each developer additionally needs:

1. **Git** and a GitHub account with access to the team repo.
2. **A code editor** — VS Code recommended, with the "Dev Containers" extension (lets you open the whole project *inside* the Docker container, so nobody fights Python/Node version mismatches on their host machine).
3. **An Anthropic API key** (or your team's chosen model provider key) — one shared key stored in a password manager, never in code.
4. **Docker Compose v2** — bundled with Docker Desktop already; confirm with `docker compose version` in a terminal.
5. Familiarity with a terminal (`cd`, `ls`, running a command) — that's the only Linux skill required; the Ubuntu container is set up for you in section 3.

No one needs LangGraph, Postgres, or vector-database experience going in — section 5 explains the concepts you need before Dev 1 starts the orchestrator.

1. Repo layout

One repo, one Docker Compose stack, one folder per module so the ten agents never collide on files:

```
fcrm-workbench/
├── docker-compose.yml           # IaC — the whole stack, one command to stand up
├── .env.example                 # template for secrets (copy to .env, never commit .env)
├── Dockerfile                   # Ubuntu 22.04 app image — every module
```

```
runs inside it
├── requirements.txt              # shared Python deps (LangGraph, LangChain,
FastAPI, etc.)
├── infra/
│   ├── init-db/                  # SQL run once when Postgres container
first starts
│   │   └── 001_schema.sql
│   └── mock-services/            # synthetic screening + policy seed data
├── app/
│   ├── orchestrator/              # M0 — Dev 1
│   ├── intake/                    # M1 — Dev 3
│   ├── extraction/                # M2 — Dev 2
│   ├── retrieval/                 # M3 — Dev 2
│   ├── screening/                 # M4 — Dev 2 + Dev 3
│   ├── scoring/                   # M5 — Dev 1
│   ├── drafting/                  # M6 — Dev 1
│   ├── gates/                     # M7 — Dev 1 + Dev 3
│   ├── eval/                      # M8 — Dev 4
│   ├── notifications/             # M9 — Dev 3
│   └── ui/                        # Workbench UI — Dev 3
├── tests/
└── docs/                          # architecture diagrams, governance docs
```

Every module is a plain Python package with its own `agent.py` (the LangGraph node or tool logic) and `schema.py` (Pydantic input/output types) — this is what keeps the ten "agents" from turning into one tangled file.

2. Infrastructure as code — the Docker stack

One `docker-compose.yml` at the repo root defines the whole system as code. Anyone on the team runs `docker compose up` and gets: the Ubuntu app container (all ten modules), Postgres with pgvector (case DB + policy library), and Langfuse (eval tracing). Nothing is clicked together by hand — that IS the "infrastructure as code" requirement.

2.1 `Dockerfile` — the Ubuntu app image

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONUNBUFFERED=1
```

```
# System packages every module needs: Python, build tools, Postgres client
libs
RUN apt-get update && apt-get install -y \
    python3.11 python3.11-venv python3-pip \
    build-essential libpq-dev curl git \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.orchestrator.main:app", "--host", "0.0.0.0", "--port",
"8000", "--reload"]
```

This one image runs all ten modules — they're Python packages imported by the orchestrator, not separate containers — which keeps the stack simple enough to still score well on "engineering judgment" and "token/infra efficiency."

2.2 `requirements.txt` — the shared toolchain

```
langgraph>=0.2
langchain-anthropic
fastapi
uvicorn[standard]
pydantic>=2
sqlalchemy
psycopg2-binary
pgvector
voyageai
ragas
deepeval
langfuse
python-multipart
python-dotenv
```

2.3 `docker-compose.yml` — the full stack as code

```
services:
  app:
    build: .
    container_name: fcrm-app
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
      - db
    volumes:
      - ./app:/app/app          # live-reload during development
    networks: [fcrm-net]

  db:
    image: pgvector/pgvector:pg16   # Postgres 16 + pgvector extension pre-
installed
    container_name: fcrm-db
    environment:
      POSTGRES_USER: fcrm
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: fcrm_workbench
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./infra/init-db:/docker-entrypoint-initdb.d   # schema.sql runs on
first boot
    ports:
      - "5432:5432"
    networks: [fcrm-net]

  langfuse:
    image: langfuse/langfuse:2
    container_name: fcrm-langfuse
    environment:
      DATABASE_URL:
postgresql://fcrm:${POSTGRES_PASSWORD}@db:5432/fcrm_workbench
      NEXTAUTH_SECRET: ${LANGFUSE_SECRET}
      SALT: ${LANGFUSE_SALT}
    ports:
      - "3001:3000"
    depends_on: [db]
    networks: [fcrm-net]
```

```
volumes:
  db-data:

networks:
  fcrm-net:
```

This is genuinely infrastructure-as-code: the whole environment (app + database + observability) is declared in version-controlled files, reproducible on any developer's machine with one command, and it's what you'll point to in the deployment/production-readiness section of the submission.

2.4 Standing it up

```
git clone <repo-url> && cd fcrm-workbench
cp .env.example .env        # fill in your API key and a password
docker compose up --build   # first run: builds the image, starts db +
langfuse + app
```

Visit `http://localhost:8000/docs` for the app's auto-generated API docs and `http://localhost:3001` for the Langfuse trace dashboard once M8 is wired up.

3. Secrets & environment variables

Create `.env.example` in the repo (committed) and each developer copies it to `.env` (git-ignored):

```
# .env.example
ANTHROPIC_API_KEY=sk-ant-...
POSTGRES_PASSWORD=change-me
DATABASE_URL=postgresql://fcrm:change-me@db:5432/fcrm_workbench
LANGFUSE_SECRET=generate-a-random-string
LANGFUSE_SALT=generate-another-random-string
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
```

Add `.env` to `.gitignore` on day one. Since screening (M4) and policy data (M3) are entirely synthetic, no real vendor API keys are ever needed — the only real secret in this project is the model provider key.

4. Concept primer — what a LangGraph "agent" actually is

For developers who haven't built agents before, three ideas cover everything needed for this project:

- **A node is just a Python function.** It takes the current case state in, calls Claude (or plain code) to do one job, and returns updated state. M2 (document extraction) is a node; M5 (risk scoring) is a node. Nothing magical — no node needs to know about any other node.
- **A `StateGraph` wires nodes into a flow.** You define a shared state object (the case: its facts, citations, score, memo, status) and connect nodes with edges — including edges that fan out to run several nodes in parallel (M2, M3, M4 all run at once off the same state) and edges that are conditional ("if screening found a hit, go here; otherwise, go there").
- **`interrupt()` is the human gate.** Calling `interrupt()` inside a node pauses the whole graph and returns control to a person; the graph does not resume until someone calls `resume()` with their decision. This single primitive is what makes Gate 1 and Gate 2 real stops instead of decorative UI — nothing can be coded around it because the graph literally cannot proceed without that call.

Every module below is one node (or a small group of nodes) built and tested in isolation before it's wired into the graph in section 9.

5. Build step 1 — M1 Intake & Validation (Dev 3, first working agent)

Build M1 first and in isolation — it needs nothing from any other module and gives the team a working end-to-end slice (form in, validated case out) to demo early.

1. **Define the case schema** in `app/intake/schema.py` with Pydantic — product type, change type, geography, segment, target date, uploaded file references. This schema becomes the shared "state" every later module reads and writes.
2. **Write the validation node** in `app/intake/agent.py`: a plain function that checks required fields, and one Claude Haiku call that classifies the change type / flags anything ambiguous ("is this a new product or a feature change?"). Keep the prompt short — Haiku here is a triage step, not a reasoning step.
3. **Assign a case ID** and write the record into the `cases` table in Postgres (schema goes in `infra/init-db/001_schema.sql`; run automatically the first time the `db` container boots).

4. **Return a `needs-more-information` event** instead of a validated case when required fields are missing — this is the shape M7's RFI loop will reuse later, so get the event contract right now.
5. **Test it standalone**: a small FastAPI route (`POST /cases`) that accepts a form + files and returns the validated case record, runnable with `docker compose up app` before any other module exists.

Deliverable: a case can be submitted through the API and lands in Postgres with a case ID — this is the seed the whole graph builds around.

6. Build step 2 — M2, M3, M4 (Dev 2 + Dev 3, built in parallel, wired later)

These three each take the validated case from M1 and add one kind of enrichment. Build and unit-test each against a hard-coded sample case *before* touching the graph — they don't depend on each other.

M2 — Document Extraction (`app/extraction/`, Dev 2)

- Accept the uploaded PDFs/DOCX/images from M1.
- Use Claude's document understanding (send the file directly in the message) to pull out parties, jurisdictions, customer types, transaction types.
- Every extracted field must carry a source page reference — return `{"field": ..., "value": ..., "source_page": ...}`, not bare values, so M6's citations have something to point to.

M3 — Policy & Regulatory Retrieval (`app/retrieval/`, Dev 2)

- Author the synthetic policy library first (a handful of markdown/text policy documents covering the change types you expect to demo, e.g. cannabis banking, new product launches).
- Embed them with Voyage embeddings into the `db` container's pgvector extension (`CREATE EXTENSION vector;` — already available via the `pgvector/pgvector:pg16` image).
- Retrieval node: embed the case facts, query the top-k policy chunks, optionally rerank with Cohere, return ranked citations + comparable past cases.

M4 — Screening (`app/screening/`, Dev 2 + Dev 3)

- Build a tiny mock REST service (can live in the same FastAPI app under `/mock/screening`) backed by a synthetic watchlist table — a few dozen fake sanctioned names/entities is enough.
- For each named party from M2, check for exact and fuzzy matches; ambiguous matches go through one Claude Haiku triage call to produce hit / no-hit / needs-review with a confidence score.

Each of these three is a single LangGraph node function for now — they get fanned out to run concurrently once M0 exists in section 9.

7. Build step 3 — M5 Risk Scoring and M6 Draft Assessment (Dev 1)

M5 — Risk Scoring (`app/scoring/`)

- Write the scorecard as **plain, versioned Python** first — a dict of weighted factors (jurisdiction risk, cash intensity, screening hits, product novelty) that sums to a tier (Low/Med/High/Critical). This is deterministic code, not an LLM call — the rubric explicitly rewards a score the model explains rather than invents.
- Add one Claude call afterward whose only job is to write the plain-language rationale for a score that's already been computed — pass it the factor breakdown and ask it to explain, not to decide.
- Version the scorecard file (`scorecard_v1.py`) so the eval harness (M8) can regression-test against a fixed version even as you tune weights.

M6 — Draft Assessment (`app/drafting/`)

- One Claude call with long context: feed it the outputs of M2–M5 (case facts, citations, screening report, score + rationale) and a versioned memo template.
- Prompt it to produce a structured memo — rating, citations (linked back to M2's source pages and M3's policy citations), open questions, suggested conditions — as JSON or structured markdown, not free text, so the UI can render it consistently.
- This is the artifact Gate 1's analyst actually reviews, so favor completeness and traceable citations over prose style.

8. Build step 4 — M0 Orchestrator + M7 Human Review Gates (Dev 1 + Dev 3)

This is where the individually-tested modules become one system. Build it last, once M1–M6 each work standalone.

```
flowchart LR
  A[M1 Intake] --> B[M2 Extraction]
  A --> C[M3 Retrieval]
  A --> D[M4 Screening]
  B --> E[M5 Scoring]
  C --> E
  D --> E
  E --> F[M6 Draft Memo]
  F --> G{Gate 1<br/>Analyst}
  G -- RFI --> A
  G -- Finalize --> H{Gate 2<br/>Committee}
  H --> I[Closed & Archived]
```

Steps to wire it:

1. **Define the shared graph state** (`app/orchestrator/state.py`) — one `TypedDict` /Pydantic model holding everything M1–M6 read or write: case facts, citations, screening report, score, memo, status.
2. **Register each module as a node** in `app/orchestrator/main.py` using LangGraph's `StateGraph(State)` — `graph.add_node("extract", extraction_node)` and so on for all nine modules besides M0 itself.
3. **Fan out M2, M3, M4** from M1 with parallel edges so they run concurrently against the same case (they don't depend on each other's output).
4. **Fan in to M5** — LangGraph waits for all three enrichment nodes before running the scoring node.
5. **Add Gate 1**: after M6, a node that calls `interrupt({"memo": draft_memo})`. The graph pauses here. The Workbench UI's analyst review screen calls `resume({"decision": "finalize"})` or `resume({"decision": "rfi", "notes": ...})` — the latter routes back to M1's intake node as a request-for-information event.
6. **Add Gate 2**: same `interrupt()` pattern after committee review; `resume()` carries the recorded decision — Approved, Rejected, Deferred, or Approved with Conditions.
7. **Persist every gate action to an `approvals` table** (actor, timestamp, decision) — this table is what proves "nothing self-approves" during judging and what an auditor would query later.
8. **Test the gate, not just the happy path**: write an integration test that asserts the graph genuinely halts at `interrupt()` and cannot be resumed without an explicit decision — this doubles as governance evidence for the submission.

9. Build step 5 — M8 Eval/Observability and M9 Case & Notification (Dev 4 / Dev 3)

M8 — Eval & Observability Harness (`app/eval/`, Dev 4)

- Wrap the Langfuse SDK around every module call (`langfuse.trace(...)`) so every case run produces a searchable trace by case ID — the `langfuse` service in the Docker stack (section 2) gives you the dashboard for free.
- Build a golden-case set: 5–10 hand-authored sample cases with known-correct extraction, screening, and scoring outputs.
- Write a CI job (`tests/eval_regression.py`) using Ragas or DeepEval that reruns the graph against the golden set on every change and fails the build if extraction accuracy, retrieval relevance, or memo faithfulness drops below a threshold.

M9 — Case & Notification Service (`app/notifications/`, Dev 3)

- A lightweight state-transition listener on the orchestrator's events (submitted, module-completed, gate-passed, decision-recorded).
- Track an SLA timer per case against the 2-day target and flag breaches.
- Fire a templated notification (email/webhook stub — synthetic, no real inbox needed) to the relevant persona at each transition; this table is also what the Workbench UI's dashboard reads to show case status.

10. Running the full stack and smoke-testing it

```
docker compose up --build       # app on :8000, Postgres on :5432, Langfuse
on :3001
```

Smoke-test checklist, once all ten modules are wired in:

- [ ] `POST /cases` with a sample form + docs returns a case ID (M1)
- [ ] Case facts, citations, and screening results all populate for that case ID (M2, M3, M4 ran in parallel)
- [ ] A risk tier and draft memo are produced with inline citations (M5, M6)
- [ ] The graph halts at Gate 1 and does not proceed without a `resume()` call (M7)
- [ ] An RFI decision at Gate 1 correctly routes the case back to intake (M0 + M7)
- [ ] The graph halts again at Gate 2 and a recorded decision closes the case (M7, M9)

- [ ] A trace for the full case run appears in the Langfuse dashboard at `localhost:3001` (M8)
- [ ] The golden-case regression job passes in CI (M8)

Once this checklist passes end to end, the team has a working demo of every module in the blueprint — polish, UI, and rehearsal (see the program schedule) come after.

11. Minimal placeholders for M0–M9 + UI

Before any module has real logic, `uvicorn` (or Docker) needs the *files and packages to exist* and import cleanly — that's what broke in the "Could not import module" error. Run this once, from the repo root, to scaffold every module as an empty-but-importable stub. Each developer then fills in their own module's `agent.py` / `schema.py` without anyone else being blocked.

```
#!/usr/bin/env bash
set -e

MODULES=(orchestrator intake extraction retrieval screening scoring drafting
gates eval notifications ui)

mkdir -p app tests
touch app/__init__.py

for m in "${MODULES[@]}"; do
  mkdir -p "app/$m"
  touch "app/$m/__init__.py"
done

echo "Scaffolded: ${MODULES[*]}"
```

Save that as `scripts/scaffold.sh`, `chmod +x scripts/scaffold.sh`, then run `./scripts/scaffold.sh`.

`app/orchestrator/main.py` — the FastAPI entrypoint `uvicorn` needs

```
from fastapi import FastAPI

app = FastAPI(title="FCRM Workbench")


@app.get("/health")
def health():
    return {"status": "ok"}
```

This alone is enough for `uvicorn app.orchestrator.main:app --reload --port 8000` to boot and `http://localhost:8000/health` to respond — before M0's actual LangGraph wiring exists.

`app/orchestrator/state.py` — the shared case state every module will read/write

```
from typing import TypedDict, Any


class CaseState(TypedDict, total=False):
    case_id: str
    facts: dict[str, Any]
    citations: list[dict]
    screening_report: dict
    score: dict
    memo: dict
    status: str
```

One stub per module (M1–M9)

For every module folder besides `orchestrator` and `ui`, create a matching `agent.py` and `schema.py` — the pattern is identical, only the function name changes:

```
# app/<module>/schema.py  (e.g. app/intake/schema.py)
from pydantic import BaseModel


class Input(BaseModel):
    case_id: str


class Output(BaseModel):
    case_id: str
    status: str = "not_implemented"
```

```
# app/<module>/agent.py  (e.g. app/intake/agent.py)
from .schema import Input, Output


def run(payload: Input) -> Output:
    """TODO: replace with the real M1 (Intake & Validation) logic."""
    return Output(case_id=payload.case_id)
```

Repeat for `extraction` (M2), `retrieval` (M3), `screening` (M4), `scoring` (M5), `drafting` (M6), `gates` (M7), `eval` (M8), and `notifications` (M9) — swap the module name in the import comment and the docstring so it's clear which blueprint module each stub becomes. Each developer's first real commit on their module replaces the stub body, never the file's shape, so the orchestrator's imports keep working throughout.

`app/ui/` — placeholder Workbench UI

The UI is a separate frontend (Dev 3), so its stub is just a static placeholder rather than a Python module:

```
mkdir -p app/ui/static
cat > app/ui/static/index.html <<'EOF'
<!doctype html>
<html><body><h1>FCRM Workbench — coming soon</h1></body></html>
EOF
```

Mount it from `main.py` once it exists:

```
from fastapi.staticfiles import StaticFiles

app.mount("/ui", StaticFiles(directory="app/ui/static", html=True),
name="ui")
```

Sanity check

```
python -c "import app.orchestrator.main"   # should print nothing (success)
uvicorn app.orchestrator.main:app --reload --port 8000
curl http://localhost:8000/health          # → {"status":"ok"}
```

With this in place, every developer can start building their own module (sections 5–9) against a repo that already imports and runs — nobody is blocked waiting on M0's real orchestrator logic.